

# Grounding Agent-Driven Code Optimization in Production Telemetry

Piotr Bejda<sup>1</sup>, Junaid Ahmed<sup>1,\*</sup>

<sup>1</sup>Datadog, New York, USA

## Abstract

Closed-loop LLM optimizers are starting to match or exceed hand-tuned heuristics for systems code [1, 2, 3], fitting within a broader effort to redesign data systems around agentic workloads [4]. The weak point is the benchmark: an agent minimizing a synthetic benchmark tunes for whatever distribution the benchmark exercises, whether or not that matches production. We describe DODO (Datadog Observability-Driven Optimizer), a system that closes this gap by grounding the benchmark in live production telemetry. A benchmark agent reads two signals—a CPU profile from Datadog Continuous Profiler and samples of real production calls from Datadog Live Debugger—and iterates a Go micro-benchmark until its execution shape matches production at  $\geq 98\%$  similarity. A simple optimization agent then uses the benchmark as a scoring function for code changes. We deploy the system against eight hot paths in a Datadog metrics-intake service and report speedups between 4% and 82%. Three of these optimizations have been validated in production, together cutting over 8% of the service’s total CPU cost.

## Keywords

LLM agents, code optimization, production telemetry, benchmarking, continuous profiling, Go

## 1. Introduction

We close the gap between synthetic micro-benchmarks and real production workloads with two observability signals, fed to an LLM agent that writes a faithful benchmark.

We use Datadog Live Debugger to dynamically attach a probe to the target function in the running production service. The probe samples invocations across all running instances, capturing for each the function’s inputs, outputs, and execution state. Sampling across many instances yields a diverse set of real-world examples. We want variety in the kinds of inputs that impact CPU cost differently; statistical significance is not required, since the agent later fits the weight of each kind to match the production profile.

We also collect a CPU profile of the live service via Datadog Continuous Profiler. Running across all production instances, it captures how CPU time is spent inside the target function during execution—how much in the function itself, and how much in each subroutine it calls, all the way down.

These two signals combine to define what a faithful benchmark looks like. The data captured by the debugger seeds a set of realistic test cases—concrete examples of how the function is invoked in production. The profile defines the execution shape the benchmark must reproduce when it runs those test cases: the relative time spent in each branch of the call tree. A benchmark agent draws on both—the test cases to write the benchmark’s setup, and the profile as both target and feedback signal, scoring each candidate benchmark’s own profile against production’s. Grounding the benchmark in production telemetry along both axes—as the source of test cases and as the target execution shape—is the main contribution of the paper.

Once the benchmark exists, the optimization step is comparatively straightforward. Prior work has explored sophisticated synthesis and evolutionary loops for systems-code

optimization [2, 3]; we found that a simple LLM agent—read the code (including realistic benchmark data), propose a change, run the tests, run the benchmark, repeat—produces strong results without elaborate scaffolding. This likely reflects how much progress recent LLMs have made at organizing their own work.

We implement this approach in DODO, targeting Go services, but the design is not language-specific.

## 2. Architecture

DODO consists of two loops: benchmark generation and code optimization.

### 2.1. Loop 1—production-grounded benchmark generation

The benchmark agent runs an iterative loop, sketched in Figure 1: read the two production signals, write a candidate benchmark, run it, score its CPU profile against production’s, and use the divergences as feedback for the next iteration. The loop terminates when the similarity threshold is met or the turn budget (the maximum number of agent iterations allowed) is exhausted.

The benchmark agent is given a target function, a benchmark file path, and two streams of production signal:

- **A pruned CPU call tree rooted at the target function.** We fetch an aggregated flamegraph from the Datadog profiling API for the target service, filtered by the CPU architecture, walk to the target function, and prune subtrees contributing less than 1%. This tree is included in the system prompt and used as ground truth by the evaluation tool.
- **Real invocations from live traffic.** The agent places Live Debugger probes to capture production invocations of the target, including arguments and receiver state.

Capturing the receiver state is as important as capturing the arguments. Many hot-path targets carry elaborate internal configurations—rule sets, pre-populated caches, lookup tables—whose construction from setup code is time-consuming and brittle to reproduce. Direct state capture lets

*Supporting Our AI Overlords Workshop (SAO) at the ACM Conference on AI and Agentic Systems (CAIS 2026), May 26, 2026, San Jose, California, USA*

\*Corresponding authors.

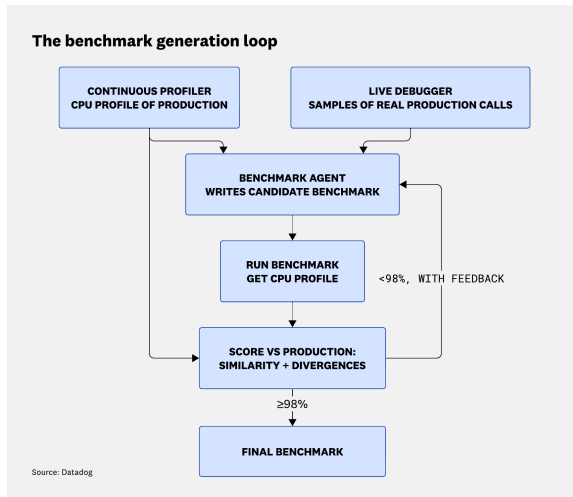
<sup>†</sup>These authors contributed equally.

✉ piotr.bejda@datadoghq.com (P. Bejda);

junaaid.ahmed@datadoghq.com (J. Ahmed)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).



**Figure 1:** The benchmark generation loop. The agent iterates until the benchmark’s CPU profile matches production at  $\geq 98\%$  similarity.

the benchmark agent assemble semantically valid test cases in the first place, and inherit realistic call characteristics as a side effect.

Profile similarity is only meaningful if both sides are measured on the same hardware. Two normalizations help: architecture-specific function names are aliased to a canonical form, and internal frames within Go runtime functions are collapsed to their top frame—enough to identify the operation without comparing implementation depths that differ by architecture. Raw CPU costs, however, cannot be normalized away: an amd64 hash instruction and its arm64 equivalent have different latencies, and no amount of input tuning by the agent can close that gap. We sidestep the problem by fetching the production profile with a CPU-architecture filter and running the benchmark on matching hardware. Removing this hardware-parity requirement—normalizing away per-instruction cost differences, or measuring at simulated parity—remains an open problem.

Given these inputs, the agent writes a single Go benchmark function. The `evaluate_benchmark` tool compiles it, runs it three times with CPU profiling, parses the resulting profile, prunes it the same way as the production tree, and computes a similarity score. The score is a convex combination of (a) L2 similarity on the self-plus-children weight vector at each node and (b) a weighted recursive similarity on shared children. The tool returns the score plus the top divergences—labeled *missing*, *extra*, *over*, or *under*—as absolute percentages of total profile time, so the agent can see exactly which call paths its benchmark over- or under-exercises.

### 2.1.1. Shaping agent behavior

A few engineering choices around prompts and tools matter as much as the production signals themselves. Tools should be scoped to a complete unit of work, exposed as single high-level operations rather than their constituent steps—capturing production calls, for example, involves placing a probe, waiting, retrieving the data, and tearing it down, but the agent calls a single `capture_production_data` tool that handles all four. An earlier version that surfaced each step separately consumed 5–10 agent turns on bookkeeping.

Context the agent will need but does not choose belongs in the prompt rather than in the tool surface—information known up front (the target function’s calling convention, the production deployment we instrument, the target’s location in the call tree) is injected into the system prompt rather than fetched on demand, eliminating a class of “agent goes looking for things” failure modes. Prompts describe affordances rather than procedures: telling the agent what it can do, not what to do when. The agent constructs an optimal hill-climbing path by exploring the surface through progressive planning, improving in each step. More prescriptive prompts caused agents to treat the procedure as load-bearing and get stuck when a step did not apply. Similarly, we do not impose explicit planning scaffolding—no separate planning phase, scratchpad, or task-list decomposition. Modern LLMs handle implicit planning well enough that adding such scaffolding tends to constrain rather than help.

## 2.2. Loop 2—optimization against the generated benchmark

The optimization agent receives the frozen benchmark from Loop 1 and standard code-reading and editing tools, plus `run_tests` and `run_benchmark`. Before the loop starts, the tests are run three times to fingerprint pre-existing flakes, and the benchmark is run once to establish a baseline ns/op and CPU profile; both are surfaced in the system prompt.

Each `run_benchmark` call re-runs the same command as the baseline, compares ns/op against the baseline, and snapshots the current code change as a numbered patch. The snapshot with the lowest ns/op so far is retained as the best observed state, so a regressing final edit does not overwrite earlier gains.

## 2.3. Keeping the two loops decoupled

**Disjoint write access.** The benchmark agent only writes the benchmark file; the optimization agent only writes service code and reads the benchmark. The benchmark is fixed before the optimizer starts, so it cannot drift under optimization.

**Feedback is dense, not scalar.** `evaluate_benchmark` returns the similarity score and the top divergences as a sorted list of (path, prod%, bench%, gap) tuples. In practice, this is what lets the agent close gaps in one or two iterations: “callee X is 12% in production and 2% here—I need more inputs that trigger the X branch.”

## 3. Results

We evaluated DODO on a mature Datadog metrics-intake service written in Go. The service ingests and processes the metric data Datadog receives from customer agents. After years in production at scale and many rounds of performance tuning by experienced engineers, the hot paths in this service are not new code—they had already been examined and optimized by humans before DODO ran against them.

We deployed DODO against eight such hot paths, running on arm64 (Graviton). The targets range from 1.1% to 11.5%

**Table 1**

Per-target speedup and the optimization the agent found. “Prod” marks optimizations validated in production.

| Target               | CPU%  | Speedup | Prod |
|----------------------|-------|---------|------|
| intern               | 9.8%  | 40%     | ✓    |
| MergeTags            | 11.5% | 4%      |      |
| NormalizeTags        | 7.5%  | 22%     |      |
| HandleFromSortedTags | 3.7%  | 15%     |      |
| ComputeTagsHash      | 3.2%  | 27%     |      |
| FilterPayloads       | 2.7%  | 75%     | ✓    |
| writeTagsetsMut      | 2.0%  | 76%     | ✓    |
| filterTags           | 1.1%  | 82%     |      |

of service CPU. Table 1 summarizes the speedups measured against the generated benchmarks.

Two vignettes illustrate the kind of change the optimizer produced.

**FilterPayloads.** The (\*StringerFilter) receiver carries an elaborate production configuration of filter rules—a mix of literal, prefix, and regex matchers—that the benchmark agent captured directly from live traffic rather than reconstructing from the service’s dense setup code. The baseline profile showed 86.5% of time in `Filters.Matches`, a linear scan through all filter rules. The agent observed that most rules are exact-string matches, introduced an `IndexedFilters` type backed by a map for  $O(1)$  literal lookup, and kept a linear pass only for regex and prefix matchers. The new type was threaded through four files across two packages.

**writeTagsetsMut.** The baseline showed 59% of time in `sort.Sort` (pdqsort on `uint32` tag IDs). The agent observed that tag IDs are bounded to a small integer range and replaced the sort with a bitset: set a bit for each ID, then iterate via `TrailingZeros64`.  $O(n)$  instead of  $O(n \log n)$ , plus inlined `streamvbyte` encoding with direct indexed writes.

The role of production grounding is visible in `NormalizeTags`: captured invocations revealed that roughly 25% of tags contain uppercase characters (timestamps containing T, version strings containing RC). The benchmark agent preserved that ratio; the optimization agent then found a fast ASCII case-fold path whose payoff depends on exactly that distribution. A synthetic benchmark would not have surfaced the opportunity.

## 4. Future Work

We plan to focus on the developer lifecycle in the agentic world, finding ways in which production context aids pre-production validation and post-deployment analysis. Several directions look promising:

1. Datadog Continuous Profiler and Datadog Live Debugger as direct validation signals
2. Production prompts and inputs in the formation of unit, acceptance, and synthetic tests
3. For LLM applications, using production prompts for local inline CI or experimental validation
4. Post-release trace aggregates and metric anomalies for service hardening
5. Post-release session analysis for business success and user journey monitoring

## Acknowledgments

This work would not have been possible without the endless support from the Debugger, Profiling, and Metrics Intake teams. In particular, Andrew Werner and Andrei Matei have closely worked on making this approach feasible.

## References

- [1] A. Cheng, S. Liu, M. Pan, Z. Li, B. Wang, A. Krentsel, T. Xia, M. Cemri, J. Park, S. Yang, J. Chen, L. Agrawal, A. Desai, J. Xing, K. Sen, M. Zaharia, I. Stoica, Barbarians at the gate: How AI is upending systems research, 2025. URL: <https://arxiv.org/abs/2510.06189>. `arXiv:2510.06189`.
- [2] A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog, AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL: <https://arxiv.org/abs/2506.13131>. `arXiv:2506.13131`.
- [3] J. Lao, I. Trummer, GenDB: The next generation of query processing—synthesized, not engineered, 2026. URL: <https://arxiv.org/abs/2603.02081>. `arXiv:2603.02081`.
- [4] S. Liu, S. Ponnappalli, S. Shankar, S. Zeighami, A. Zhu, S. Agarwal, R. Chen, S. Suwito, S. Yuan, I. Stoica, M. Zaharia, A. Cheung, N. Crooks, J. E. Gonzalez, A. G. Parameswaran, Supporting our AI overlords: Redesigning data systems to be agent-first, 2025. URL: <https://arxiv.org/abs/2509.00997>. `arXiv:2509.00997`.